FOLLOW ALONG PDF

Polymarket AI Trading Guide

Commands, Claude Code prompts, dashboard prompts, and safety checklist in exact video order.

Sharbel | Bullpen | Claude Code | Polymarket

Use it like this	Safety rule
Go top to bottom during filming. Copy each command or prompt at the matching timestamp.	Dry-run first. Live mode only after max position size, daily loss, spread, liquidity, blocked markets, and emergency stop are set.

Contents

- Setup commands
- Claude Code setup prompt
- Strategy 1 to 5 automatic agent prompts
- Alert templates
- Dashboard prompts
- Final live-mode checklist

BULLPEN SETUP, 4:00

1. Terminal Command

BASH

```
brew install bullpenfi/tap/bullpen
```

2. Terminal Command

BASH

```
bullpen skill install
```

3. Terminal Command

BASH

```
bullpen login
```

4. Terminal Command

BASH

```
bullpen status
```

5. Terminal Command

BASH

```
bullpen polymarket discover --sort volume
```

6. Terminal Command

BASH

```
bullpen polymarket search "bitcoin"
```

7. Terminal Command

BASH

```
bullpen polymarket price <slug>
```

8. Terminal Command

BASH

```
bullpen polymarket positions
```

9. Terminal Command

BASH

```
bullpen polymarket buy <slug> Yes 10 --preview
```

10. Claude Code Prompt

TEXT

```
You are my Bullpen setup assistant. Help me verify Bullpen is installed, logged in,
connected to Polymarket, and able to run read-only market discovery commands. Do not
execute any trade. If auth, balance, CLI, or API checks fail, stop and tell me exactly
what failed.
```

STRATEGY 1, COPY TRADING SMART MONEY, 8:00

11. Claude Code Prompt

TEXT

```
You are building an automatic Polymarket smart money copy trading agent for me inside
Claude Code.

Goal:
Create an agent that runs on a schedule, tracks smart Polymarket wallets, detects new
trades worth copying, filters out bad copy trades, and sends me a concise alert when
there is a real opportunity.

Use Bullpen as the Polymarket data layer if available. You may use local files to store
watchlists, wallet history, previous alerts, and agent state. Do not expose private
keys, session tokens, wallet secrets, or balances. Default to dry-run first, then
support automatic execution only after I explicitly enable live mode. In live mode,
place trades automatically only when the strategy rules, liquidity filters, spread
limits, max position size, and daily loss limits are all satisfied. Never expose
private keys, session tokens, wallet secrets, or balances.

Agent requirements:
1. Find candidate smart money wallets from leaderboards, large trades, and followed
wallets.
2. Track each wallet by category, recent performance, market type, average position
size, and repeat behavior.
3. Detect new trades from tracked wallets.
4. For each new trade, check current price, entry timing, liquidity, spread, market
volume, and whether I am late.
5. Classify every opportunity as copy candidate, watchlist, or skip.
6. For copy candidates, explain the thesis, risk, invalidation trigger, and suggested
max position size.
7. Save every alert so the agent does not spam duplicates.
8. Produce a daily summary and urgent alerts only when a high-quality copy candidate
appears.
9. Include a dry-run mode first so I can test the agent without placing trades.

Build the first version now. Show me the file structure, the schedule, the data it
stores, and one example alert using sample or live read-only data. Build dry-run mode
first, then show me exactly how to enable live auto-execution safely with max position
size, max daily loss, allowed markets, blocked markets, and emergency stop.
```

12. Copy Paste Template

TEXT

```
SMART MONEY COPY TRADE ALERT
Wallet: [redacted]
Market: [market title]
Side: Yes
Wallet entry: 42c
Current price: 48c
Liquidity: good
Spread: 2c
Status: copy candidate
Why it matters: wallet has repeated edge in this category and entered before volume
spike.
Risk: market resolves against the assumed catalyst.
Action: preview only, human confirmation required.
```

STRATEGY 2, CRYPTO SHORT-WINDOW MARKET AGENT, 16:00

13. Claude Code Prompt

TEXT

```
You are building an automatic Polymarket crypto short-window market agent for me inside
Claude Code.

Goal:
Create an agent that monitors active Bitcoin and Ethereum short-window markets, filters
for tradable setups, compares Polymarket prices against external crypto market data,
and alerts me only when a market is worth reviewing.

Use Bullpen as the Polymarket data layer if available. Use reliable external price data
for BTC and ETH. You may store market history, alerts, and watchlists locally. Do not
expose private keys, tokens, wallet secrets, or balances. Default to dry-run first,
then support automatic execution only after I explicitly enable live mode. In live
mode, place trades automatically only when the setup passes every filter, including
liquidity, spread, time remaining, max position size, and daily loss limit. Never
expose private keys, session tokens, wallet secrets, or balances.

Agent requirements:
1. Scan active BTC and ETH markets on a schedule.
2. Track market title, slug, resolution time, current Yes and No prices, volume,
liquidity, spread, and time remaining.
3. Pull external BTC and ETH price data and recent volatility.
4. Compare current Polymarket price to spot price, trend, volatility, and time
remaining.
5. Reject markets with weak liquidity, wide spreads, unclear resolution, or too little
time to act.
6. Classify each market as trade candidate, watchlist, or hard pass.
7. For trade candidates, explain the thesis, invalidation trigger, maximum acceptable
spread, and why this is not just gambling.
8. Save state so the agent does not repeat the same alert every run.
9. Send a concise alert only when a market passes the filters.
10. Include a dry-run mode first.

Build the first version now. Show me the schedule, the filter thresholds, the stored
state, and one example alert. Build dry-run mode first, then show me exactly how to
enable live auto-execution safely with max position size, max daily loss, allowed
markets, blocked markets, and emergency stop.
```

14. Copy Paste Template

TEXT

```
CRYPTO MARKET ALERT
Market: Bitcoin above [price] by [time]
Current Yes price: 54c
Time remaining: 2h 15m
Spread: 2c
Liquidity: good
BTC spot trend: upward over last 30m
Status: watchlist
Why not automatic buy: volatility is high and invalidation is close.
Action: preview only if price stays below 56c and spread stays under 3c.
```

STRATEGY 3, WEATHER MARKET AGENT, 24:00

15. Claude Code Prompt

TEXT

```
You are building an automatic Polymarket weather market research agent for me inside
Claude Code.

Goal:
Create an agent that monitors active weather-related Polymarket markets, checks them
against reliable external weather sources, compares the market implied probability to
source-backed evidence, and alerts me only when there may be a real pricing mismatch.

Use Bullpen as the Polymarket data layer if available. Use reliable weather sources
only, such as official forecasts, national weather services, NOAA, NWS, ECMWF
summaries if accessible, local meteorological agencies, or clearly cited forecast
data. Do not use random social posts as primary evidence. Do not expose private keys,
tokens, wallet secrets, or balances. Default to dry-run first, then support automatic
execution only after I explicitly enable live mode. In live mode, place trades
automatically only when source quality, liquidity, spread, resolution criteria, max
position size, and daily loss limits all pass. Never expose private keys, session
tokens, wallet secrets, or balances.

Agent requirements:
1. Scan active weather markets on a schedule.
2. Store market title, slug, resolution criteria, current price, volume, liquidity,
spread, and deadline.
3. Pull or summarize reliable external forecast data relevant to each market.
4. Compare market implied probability against the source-backed probability or
directional evidence.
5. Reject markets with vague resolution criteria, weak liquidity, wide spreads, or
unreliable external data.
6. Classify each market as trade candidate, watchlist, or hard pass.
7. For candidates, include source links, the strongest Yes case, strongest No case,
invalidation trigger, and next forecast update time.
8. Save state and avoid duplicate alerts.
9. Send alerts only when new forecast information materially changes the thesis.
10. Include a dry-run mode first.

Build the first version now. Show me the schedule, source list, filter thresholds,
stored state, and one example alert. Build dry-run mode first, then show me exactly
how to enable live auto-execution safely with max position size, max daily loss,
allowed markets, blocked markets, and emergency stop.
```

16. Copy Paste Template

TEXT

```
WEATHER MARKET ALERT
Market: [market title]
Current price: 31c
Source-backed view: elevated probability based on [source]
Next forecast update: [time]
Liquidity: acceptable
Spread: 3c
Status: watchlist
Why not automatic buy: forecast confidence is still medium and resolution criteria need
review.
Action: monitor next update, prepare preview only if price stays below 35c.
```

STRATEGY 4, ARBITRAGE AND CORRELATED MARKET AGENT, 32:00

17. Claude Code Prompt

TEXT

```
You are building an automatic Polymarket arbitrage and correlated market monitoring
agent for me inside Claude Code.

Goal:
Create an agent that scans active Polymarket events for correlated markets, logically
related outcomes, possible hedges, and potential arbitrage structures. The agent
should be skeptical by default and alert me only when a possible opportunity survives
liquidity, spread, fee, timing, and resolution checks.

Use Bullpen as the Polymarket data layer if available. Store event groups, related
markets, price history, prior alerts, and rejected opportunities locally. Do not
expose private keys, tokens, wallet secrets, or balances. Default to dry-run first,
then support automatic execution only after I explicitly enable live mode. In live
mode, execute only when the edge survives spread, fees, liquidity, slippage, timing,
resolution checks, max position size, and daily loss limits. If a split, merge, hedge,
or trade is involved, log the exact structure before execution. Never expose private
keys, session tokens, wallet secrets, or balances.

Agent requirements:
1. Scan active high-volume markets and group related markets by event, category, and
shared resolution logic.
2. Identify possible inconsistencies between related outcomes.
3. Check whether the markets actually resolve on the same underlying event.
4. Estimate whether any apparent edge survives spread, fees, liquidity, slippage, and
timing.
5. Reject anything with unclear rules, weak liquidity, or mismatched resolution
criteria.
6. Classify each finding as real candidate, thin edge, educational only, or skip.
7. For real candidates, explain the exact trade structure, what could break it, and what
needs human review.
8. Save every candidate and rejected candidate so the agent learns which patterns are
noise.
9. Send alerts only when the potential edge is above a threshold after costs.
10. Include a dry-run mode first.

Build the first version now. Show me the schedule, grouping logic, edge threshold,
stored state, and one example alert. Build dry-run mode first, then show me exactly
how to enable live auto-execution safely with max position size, max daily loss,
allowed markets, blocked markets, and emergency stop.
```

18. Copy Paste Template

TEXT

```
CORRELATED MARKET ALERT
Event: [event name]
Markets: [market A] and [market B]
Possible issue: prices imply inconsistent probabilities
Estimated edge after spread: 1.8 percent
Liquidity: medium
Resolution match: needs human review
Status: educational only
Why not automatic trade: resolution criteria may not match perfectly.
Action: review rules before previewing anything.
```

STRATEGY 5, EVENT-DRIVEN NEWS AGENT, 41:00

19. Claude Code Prompt

TEXT

```
You are building an automatic Polymarket event-driven news trading agent for me inside
Claude Code.

Goal:
Create an agent that monitors new and fast-moving Polymarket markets, checks reliable
news sources for catalysts, compares the news to market price movement, and alerts me
only when there may be a tradable information gap.

Use Bullpen as the Polymarket data layer if available. Use reliable external sources for
news verification. Store watched markets, news sources, price changes, previous
alerts, and catalyst history locally. Do not expose private keys, tokens, wallet
secrets, or balances. Default to dry-run first, then support automatic execution only
after I explicitly enable live mode. In live mode, place trades automatically only
when the catalyst is confirmed, the price has not fully repriced, liquidity and spread
pass, max position size is respected, and daily loss limits are intact. Never expose
private keys, session tokens, wallet secrets, or balances.

Agent requirements:
1. Scan newest markets and highest 24-hour volume movers on a schedule.
2. Track market title, slug, category, price, volume, liquidity, spread, holders,
comments if useful, and recent trades.
3. Search reliable news sources for fresh catalysts related to each market.
4. Separate confirmed facts from speculation, rumors, and low-quality sources.
5. Compare the timing of the news to the market price move.
6. Decide whether the catalyst is fresh, already priced in, ambiguous, or too risky.
7. Classify each market as trade candidate, watchlist, or skip.
8. For candidates, include source links, strongest Yes case, strongest No case, next
catalyst, invalidation trigger, and whether the market may have overreacted or
underreacted.
9. Save state so duplicate headlines do not create duplicate alerts.
10. Send urgent alerts only when a new catalyst appears before the market fully
reprices.
11. Include a dry-run mode first.

Build the first version now. Show me the schedule, source list, catalyst scoring logic,
stored state, and one example alert. Build dry-run mode first, then show me exactly
how to enable live auto-execution safely with max position size, max daily loss,
allowed markets, blocked markets, and emergency stop.
```

20. Copy Paste Template

TEXT

```
NEWS CATALYST ALERT
Market: [market title]
Catalyst: [confirmed news]
Source: [link]
Price move: 34c to 49c in 40 minutes
Status: watchlist
Agent view: news is real, but partially priced in
Next catalyst: [upcoming event]
Risk: headline may be overinterpreted
Action: monitor, prepare preview only if price pulls back below 45c.
```

GOOD VS BAD MARKETS, 50:00

21. Claude Code Prompt

TEXT

```
Score this Polymarket market from 1 to 10 as a trading opportunity. Evaluate resolution
clarity, volume, liquidity, spread, recent trades, data availability, catalyst timing,
smart money signal, and downside risk. Classify it as trade candidate, watchlist, or
hard pass. A good prediction is not enough. Explain whether it is a good trade.
```

KEEPING TRADING AGENTS UP TO DATE, 1:00:00

22. Claude Code Prompt

TEXT

```
You are my Polymarket trading research agent. You may research, summarize, rank, and
preview trades. You may not execute trades without explicit human confirmation. Before
every recommendation, refresh market price, liquidity, spread, recent trades,
relevant external data, my current positions, and open orders. If any data source
fails, stop and report the failure. Separate facts, assumptions, and speculation.
Always include invalidation criteria, position sizing notes, and risk notes.
```

Dashboard Prompts

Use these at the dashboard section of the video.

Dashboard Prompt 1, Smart Money Copy Trading

Copy this prompt into Claude Code for this exact dashboard.

TEXT

```
You are building a dashboard for my automatic Polymarket smart money copy trading agent.

Goal:
Create a clean operator dashboard that shows which wallets I am tracking, what trades
they are taking, which trades are copy candidates, and what the agent has done or is
preparing to do.

Use Bullpen as the Polymarket data layer if available. Use the agent's local state
files, alerts, trade logs, wallet watchlists, dry-run results, and live-mode execution
logs if they exist. Never expose private keys, session tokens, wallet secrets, or
sensitive balances.

Dashboard sections:
1. Wallet watchlist, wallet address, label, category, reason followed, confidence score,
last active time.
2. Smart money feed, latest tracked wallet trades, market, side, entry price, size,
time, and category.
3. Copy trade candidates, market, wallet entry, current price, spread, liquidity,
volume, time since wallet entry, status.
4. Copyability score, early, fair, late, or skip, with reason.
5. Risk filters, max position size, max daily loss, max spread, minimum liquidity,
blocked categories, emergency stop.
6. Agent mode, dry-run or live, last run time, next scheduled run, whether
auto-execution is enabled.
7. Recent agent decisions, copied, skipped, watched, or rejected, with reason.
8. P&L and performance log, simulated P&L for dry-run, real P&L for live mode if
enabled, win rate, average return, average hold time.
9. Alerts, urgent copy candidates, wallet behavior changes, risk limit warnings.

Build this as a simple local dashboard I can run while filming. Prefer a clean web
dashboard if fast, otherwise generate a markdown or HTML dashboard first. Use sample
data only if live data is unavailable, and clearly label sample data.

Add buttons or clear actions for: Refresh data, Add wallet, Remove wallet, Switch
dry-run/live mode, Preview trade, Enable emergency stop.

Do not execute any trade from the dashboard unless live mode is explicitly enabled and
all risk rules pass.
```

Dashboard Prompt 2, Crypto Short-Window Market Agent

Copy this prompt into Claude Code for this exact dashboard.

TEXT

```
You are building a dashboard for my automatic Polymarket crypto short-window market
agent.

Goal:
Create a dashboard that monitors BTC and ETH short-window markets, shows which markets
are tradable, which ones are junk, and what the agent is doing in dry-run or live
mode.

Use Bullpen as the Polymarket data layer if available. Use reliable external BTC and ETH
price data. Use the agent's local state, market history, alerts, dry-run logs, and
live trade logs if they exist. Never expose private keys, session tokens, wallet
secrets, or sensitive balances.

Dashboard sections:
1. Active crypto markets, market title, slug, asset, target price, resolution time,
current Yes/No price, volume, liquidity, spread.
2. External price feed, BTC/ETH spot price, recent trend, volatility, last update time,
data source.
3. Market scoring, tradable, watchlist, or hard pass, with reason.
4. Time remaining, countdown to resolution and warnings for too little time left.
5. Spread and liquidity filter, show which markets fail because spread is too wide or
liquidity is too weak.
6. Agent mode, dry-run or live, last run time, next scheduled run, whether
auto-execution is enabled.
7. Trade candidates, market, side, current price, thesis, invalidation trigger, max
entry price, max position size.
8. Recent decisions, entered, skipped, watched, rejected, with reason.
9. Risk controls, max position size, max daily loss, max open positions, max spread,
allowed assets, emergency stop.
10. Performance log, dry-run performance and live performance separately, win rate,
average return, bad entries avoided.

Build this as a simple local dashboard I can run while filming. Prefer a clean web
dashboard if fast, otherwise generate a markdown or HTML dashboard first. Use sample
data only if live data is unavailable, and clearly label sample data.

Add buttons or clear actions for: Refresh markets, Refresh BTC/ETH price data, Toggle
dry-run/live mode, Preview trade, Pause agent, Emergency stop.

Do not execute any trade from the dashboard unless live mode is explicitly enabled and
all crypto market filters pass.
```

Dashboard Prompt 3, Weather Market Agent

Copy this prompt into Claude Code for this exact dashboard.

TEXT

```
You are building a dashboard for my automatic Polymarket weather market agent.

Goal:
Create a dashboard that monitors weather-related Polymarket markets, compares market
prices to reliable forecast data, and shows when the agent has found a possible
weather pricing mismatch.

Use Bullpen as the Polymarket data layer if available. Use reliable weather sources
only, such as official forecasts, NOAA, NWS, national weather services, local
meteorological agencies, or clearly cited forecast data. Use the agent's local state,
source history, alerts, dry-run logs, and live trade logs if they exist. Never expose
private keys, session tokens, wallet secrets, or sensitive balances.

Dashboard sections:
1. Active weather markets, market title, slug, location, event type, deadline,
resolution criteria, price, volume, liquidity, spread.
2. Forecast source panel, source name, link, last update time, forecast summary,
confidence level.
3. Market vs forecast comparison, market implied probability, source-backed probability
or directional view, gap between the two.
4. Market quality score, clear resolution, reliable data, good liquidity, acceptable
spread, pass or fail each filter.
5. Candidate trades, market, side, thesis, source evidence, current price, max entry
price, invalidation trigger.
6. Next update tracker, next forecast update, next resolution milestone, when the agent
will re-check.
7. Agent mode, dry-run or live, last run time, next scheduled run, whether
auto-execution is enabled.
8. Recent decisions, entered, skipped, watched, rejected, with reason and source links.
9. Risk controls, max position size, max daily loss, max spread, minimum liquidity,
allowed weather categories, emergency stop.
10. Performance log, dry-run and live results separately, which source signals worked,
which source signals failed.

Build this as a simple local dashboard I can run while filming. Prefer a clean web
dashboard if fast, otherwise generate a markdown or HTML dashboard first. Use sample
data only if live data is unavailable, and clearly label sample data.

Add buttons or clear actions for: Refresh weather markets, Refresh forecast data, Open
source links, Toggle dry-run/live mode, Preview trade, Pause agent, Emergency stop.

Do not execute any trade from the dashboard unless live mode is explicitly enabled and
all weather source, liquidity, spread, and risk filters pass.
```

Dashboard Prompt 4, Arbitrage And Correlated Market Agent

Copy this prompt into Claude Code for this exact dashboard.

TEXT

```
You are building a dashboard for my automatic Polymarket arbitrage and correlated market
monitoring agent.

Goal:
Create a dashboard that scans related Polymarket markets, groups them by event or
outcome logic, highlights possible mispricings, and shows whether any edge survives
spread, fees, liquidity, slippage, timing, and resolution risk.

Use Bullpen as the Polymarket data layer if available. Use the agent's local state,
event groups, price history, rejected opportunities, dry-run logs, and live trade logs
if they exist. Never expose private keys, session tokens, wallet secrets, or
sensitive balances.

Dashboard sections:
1. Event groups, event name, related markets, category, shared resolution logic, last
scanned time.
2. Related market table, market title, slug, Yes/No price, volume, liquidity, spread,
deadline.
3. Possible mispricing panel, what looks inconsistent, implied probabilities, estimated
edge before costs, estimated edge after spread, fees, and slippage.
4. Resolution criteria check, do the markets actually resolve on the same underlying
event, match, partial match, unclear, or fail.
5. Edge quality score, real candidate, thin edge, educational only, or skip.
6. Trade structure preview, legs, side, size, estimated cost, estimated payout,
worst-case risk.
7. Risk controls, max position size, max daily loss, minimum edge after costs, minimum
liquidity, max spread, emergency stop.
8. Agent mode, dry-run or live, last run time, next scheduled run, whether
auto-execution is enabled.
9. Recent decisions, entered, skipped, watched, rejected, include exact reason.
10. Performance and rejected opportunities, which apparent arbitrages were fake, which
ones survived, dry-run and live performance separately.

Build this as a simple local dashboard I can run while filming. Prefer a clean web
dashboard if fast, otherwise generate a markdown or HTML dashboard first. Use sample
data only if live data is unavailable, and clearly label sample data.

Add buttons or clear actions for: Refresh event groups, Recalculate edge, Inspect
resolution criteria, Toggle dry-run/live mode, Preview trade structure, Pause agent,
Emergency stop.

Do not execute any trade from the dashboard unless live mode is explicitly enabled and
the edge survives every cost, liquidity, timing, and resolution check.
```

Dashboard Prompt 5, Event-Driven News Agent

Copy this prompt into Claude Code for this exact dashboard.

TEXT

```
You are building a dashboard for my automatic Polymarket event-driven news trading
agent.

Goal:
Create a dashboard that monitors new and fast-moving Polymarket markets, tracks
catalysts from reliable news sources, compares those catalysts to market price
movement, and shows when the agent has found a possible information gap that can be
traded automatically in live mode.

Use Bullpen as the Polymarket data layer if available. Use reliable external sources for
news verification. Use the agent's local state, watched markets, source history,
catalyst history, previous alerts, dry-run logs, and live trade logs if they exist.
Never expose private keys, session tokens, wallet secrets, or sensitive balances.

Dashboard sections:
1. New and fast-moving markets, market title, slug, category, current price, 24h volume,
liquidity, spread, recent price move.
2. Catalyst feed, confirmed news, source link, source quality, publish time, summary,
related market.
3. Price reaction panel, price before catalyst, price after catalyst, time between news
and price move, overreaction, underreaction, already priced in, or unclear.
4. Trade candidates, market, side, thesis, source links, current price, max entry price,
invalidation trigger, next catalyst.
5. Source quality filter, confirmed facts, speculation, rumor, low-quality source,
reject anything that relies on weak sources.
6. Market quality filter, clear resolution, volume, liquidity, spread, deadline, holder
concentration if available.
7. Agent mode, dry-run or live, last run time, next scheduled run, whether
auto-execution is enabled.
8. Recent decisions, entered, skipped, watched, rejected, include whether the news was
fresh, already priced in, ambiguous, or too risky.
9. Risk controls, max position size, max daily loss, max spread, minimum liquidity,
allowed categories, blocked categories, emergency stop.
10. Performance log, dry-run and live results separately, which catalysts worked, which
ones were already priced in, false positives and missed opportunities.

Build this as a simple local dashboard I can run while filming. Prefer a clean web
dashboard if fast, otherwise generate a markdown or HTML dashboard first. Use sample
data only if live data is unavailable, and clearly label sample data.

Add buttons or clear actions for: Refresh markets, Refresh news sources, Open source
links, Recalculate priced-in score, Toggle dry-run/live mode, Preview trade, Pause
agent, Emergency stop.

Do not execute any trade from the dashboard unless live mode is explicitly enabled and
the catalyst, source quality, market quality, liquidity, spread, max position size,
and daily loss rules all pass.
```

Final Safety Checklist

Run this before enabling live trading on any agent.

TEXT

```
Before enabling live mode on any agent:
1. Confirm dry-run logs look sane.
2. Set max position size.
3. Set max daily loss.
4. Set max spread.
5. Set minimum liquidity.
6. Define allowed markets and blocked markets.
7. Confirm credentials are stored safely.
8. Confirm the agent never prints private keys or tokens.
9. Test emergency stop.
10. Start with the smallest possible size.
```